Matteo LAURI - Evan STILLE - Thomas FREY - Victorien DENOYELLE

SAE 5.01

Rapport de semaine 30/10/2026

Ce qui a été fait

La semaine dernière on avait prévu d'entraîner l'IA sur la carte graphique au lieu du processeur. On a ajouté un service Docker spécial pour l'entraînement qui utilise la carte graphique de l'ordinateur, qui vérifie tout seul s'il y en a une, sinon il repasse sur le processeur. On calcule aussi en 16 bits au lieu de 32.

Grâce à la carte graphique, on a pu entraîner l'IA sur beaucoup plus de voitures. La liste est générée automatiquement par un script à partir du dataset. Dans une première version, le script fusionnait seulement les vrais doublons, par exemple une Clio en 5 portes, en break ou en RS, mais gardait les générations séparées, comme les BMW Série 3 E46 et E90. L'IA est ainsi passée de 68 à 1 002 voitures de 64 marques. Le problème, c'est que l'IA hésitait souvent entre deux générations d'une même voiture, qui se ressemblent beaucoup. On a donc modifié le script pour regrouper toutes les générations et versions d'un même modèle : une BMW M4 F82 et une M4 G82 deviennent simplement « BMW M4 ». L'IA connaît maintenant 758 voitures. Il y en a moins qu'avant, mais chacune a plus de photos, et l'IA trouve plus souvent la bonne. Les anciens identifiants restent reconnus, pour que les voitures déjà enregistrées par les utilisateurs ne soient pas perdues.

On a aussi changé de modèle d'IA pour un modèle plus puissant, EfficientNetV2, avec des images plus grandes. Avant de reconnaître la voiture, l'IA découpe maintenant la voiture sur la photo pour ne pas être gênée par le décor autour. Quand il y a plusieurs voitures sur la photo, elle prend celle au premier plan, c'est-à-dire la plus grande, alors qu'avant elle prenait parfois une voiture derrière.

Comme nos propres photos prises au téléphone étaient souvent mal reconnues, on a fait un gros entraînement d'une nuit entière, environ 5h30, sur plus de 108 000 photos. Une partie des photos est volontairement modifiée pour ressembler à des photos de téléphone, un peu floues, plus sombres ou de travers. Ce premier modèle trouvait la bonne voiture du premier coup 80,1 pourcent du temps et dans ses 3 premières propositions 88,1 pourcent du temps, contre 55 pourcent avec l'ancien modèle sur autant de voitures. Quand elle donne une réponse, elle a raison dans 93 pourcent des cas. On a aussi fait en sorte qu'un nouveau modèle ne remplace l'ancien que s'il est meilleur.

On a ensuite fait en sorte que l'IA apprenne grâce aux utilisateurs. Si l'IA se trompe, on peut appuyer sur "Ce n'est pas la bonne voiture" et choisir la bonne dans les suggestions, ou appuyer sur "Aider l'IA" pour la chercher dans la liste des 758 voitures. La fiche se met alors à jour avec la bonne voiture et la photo est gardée pour le prochain entraînement. Pour respecter le RGPD, on a ajouté dans les paramètres une option "Autoriser l'entraînement de l'IA avec mes photos", désactivée par défaut et accompagnée d'un texte qui explique ce qui est gardé. Si l'utilisateur retire son accord, ses photos sont supprimées.

Après le regroupement, on a relancé un entraînement de nuit d'environ 8h40. Cette fois, la moitié des photos restent intactes et l'autre moitié est transformée façon téléphone, pour

que l'IA apprenne aussi sur des belles images. Les photos corrigées par les utilisateurs qui ont donné leur accord sont ajoutées à l'entraînement. Sur 23 855 photos que l'IA n'avait jamais vues, elle trouve la bonne voiture du premier coup 80,8 pourcent du temps, et dans ses 3 premières propositions 87,9 pourcent du temps. Quand elle donne une réponse, elle a raison dans 92,6 pourcent des cas. Ce nouveau modèle était meilleur que l'ancien, il l'a donc remplacé automatiquement.

On a aussi mis en place GitHub Actions, à chaque envoi sur GitHub, tous les tests du backend, de l'IA et de l'application se lancent automatiquement. Enfin, on a fait un grand nettoyage du code, suppression du code qui ne servait plus, regroupement des morceaux de code en double, vérification que les images envoyées sont de vraies images. On a vérifié qu'une installation neuve du projet marche du premier coup avec une seule commande. Pour finir, on a fusionné la branche ia-integration dans main et rangé tous les rapports dans un dossier rapports.

Difficultés rencontrées

TensorFlow ne trouvait pas la carte graphique au début, il manquait le chemin d'une bibliothèque de NVIDIA, qu'on a ajouté dans la configuration Docker. Le fichier du modèle faisait aussi 173 Mo alors que GitHub refuse les fichiers de plus de 100 Mo. On l'enregistre maintenant sans les données qui servent seulement à l'entraînement, et il fait maintenant 55 Mo.

Le plus gros problème reste la différence entre les photos du dataset, qui sont des photos de presse propres, et nos photos prises au téléphone dans la rue. Par exemple une BMW M4 verte prise de trois quarts n'était pas reconnue. C'est pour ça qu'on a ajouté les photos modifiées à l'entraînement et la possibilité pour les utilisateurs de corriger l'IA. Certaines voitures restent difficiles à différencier, surtout la même voiture avant et après un restylage, comme les Volvo XC90 ou les Audi A5.

Pendant le dernier entraînement, l'ordinateur a gelé quelques minutes au début. Docker, l'entraînement et les autres logiciels ouverts demandaient plus que les 32 Go de mémoire du PC, et Windows a dû utiliser le disque à la place, ce qui est beaucoup plus lent. On a fermé les autres logiciels pendant l'entraînement.

Traçabilité de l'IA

On a demandé à l'IA de modifier le projet pour que l'entraînement utilise la carte graphique, puis d'augmenter le nombre de voitures du dataset sans regrouper des voitures différentes, et d'ajouter GitHub Actions pour lancer tous les tests automatiquement.

Quand l'application ne reconnaissait plus nos photos, on lui a demandé pourquoi. Elle a expliqué que les photos du dataset étaient trop différentes des photos de téléphone. On lui a alors demandé un modèle plus puissant, de fusionner les doublons du dataset, de découper

SAE 5.01

Rapport de semaine 30/10/2026

la voiture avant de la reconnaître, puis de préparer un gros entraînement à lancer la nuit avec une seule commande.

On lui a demandé d'analyser tout le projet pour enlever le code inutile et en double et améliorer la sécurité.

On lui a ensuite demandé de regrouper toutes les générations d'une même voiture pour qu'elle la trouve plus souvent, puis de surveiller l'entraînement de nuit toutes les 30 minutes (carte graphique, mémoire, stockage, précision) et d'éteindre Docker à la fin.

Prévu pour la semaine prochaine

Travailler sur les voitures que l'IA confond le plus.